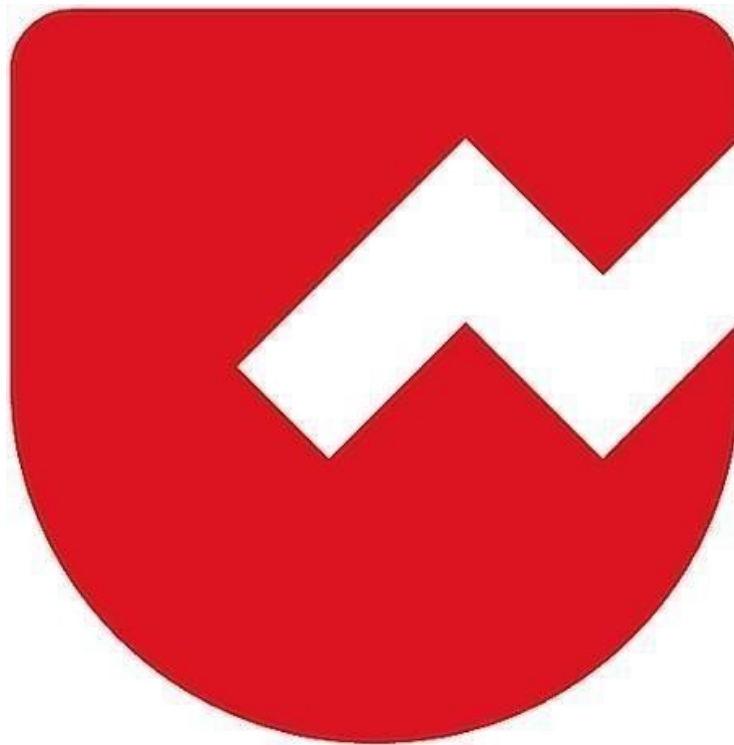




Zoho Schools for Graduate Studies



Notes

Lambda Expression

- Introduced in **Java version 8**.
- It is used to concisely represent an **anonymous** function.
- Works with **functional interfaces** (interfaces with exactly one abstract method).

Method vs Lambda Conversion:

Normal Method:

```
int add(int a, int b) {  
    return (a + b);  
}
```

Lambda Expression:

```
(int a, int b) -> {  
    return (a + b);  
}
```

Shorter form:

//No need of declare the datatype in the parameter.

```
(a, b) -> {return (a + b)};
```

Lambda Structure Explanation:

Left side -> parameter list (you need not specify datatype)

Right side -> body

Examples:

```
1. (a, b) -> {  
    return (a + b);  
};
```

```
2. (a, b) -> System.out.println(a + b);
```

//If no return statement → no need curly braces (single line).

```
//If only one statement → braces {} optional.  
3. (a) -> System.out.println(a);  
4. a -> System.out.println(a);  
    // if parameter is single, parentheses not required  
5. () -> System.out.println("ZOH0");  
    // if no parameter, parentheses mandatory
```

Method Representation:

Normal Method:

```
public static void func(int nums1, int nums2) {  
    return nums1 + nums2;  
}
```

Lambda:

```
(nums1, nums2) -> { return (nums1 + nums2); };
```

Application of Lambda Expression:

1. To implement a Functional Interface:

Functional Interface (SAM) → (Single Abstract Method):

- An interface that exactly has only one abstract method.
- It has `java.util.function`.

Examples:

1. `Runnable` -> `run()`;
2. `Comparator` -> `compare(T o1, T o2)`;
3. `Comparable` -> `compareTo(T o)`;

Functional Interface Annotation:

→ **@FunctionalInterface**

- Annotation.
- Compiler checks the method extra care.
- (If another method is added into the functional Interface class, at once it shows the error)

Functional Interface & Lambda – Example:

@FunctionalInterface

```
public interface AddInterface {  
    void add(int a, int b);  
}
```

```
public class Demo implements AddInterface {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.add(23, 25);  
    }  
    public void add(int a, int b) {  
        System.out.println(a + b);  
    }  
}
```

Another type without implements:

```
2
3 public class LambdaExpressionDemo //implements AddInterface
4 {
5     public static void main(String[] args) {
6         //LambdaExpressionDemo l=new LambdaExpressionDemo();
7         AddInterface ai=(a, b)->System.out.println(a+b);
8         ai.add(23,45);
9     }
10 }
11
12 /*public void add(int a, int b){
13     System.out.println(a+b);
14 }*/
15 }
16
```

Another example in the Threads:

```
2
3 public class ThreadDemo {
4     public static void main(String[] args) {
5         System.out.println(Thread.currentThread().getName() + " Begins!");
6         Thread.currentThread().setPriority(9);
7
8         Runnable r = ()->{
9             for(int i=1;i<=5;i++)
10                 System.out.println(Thread.currentThread().getName());
11             };
12
13         Thread t1=new Thread(r);
14         t1.setName("First");
15         System.out.println(t1.getPriority());
16         t1.start();
17     }
18 }
```

- No need of write the newThread() class that implements runnable interface.
- Directly we can write the runnable with the help of Lambda Expression.